Feedforward Neural Network

```
# Install required packages
!pip install GEOparse
!pip install scikit-learn
!pip install tensorflow
```

Requirement already satisfied: GEOparse in /usr/local/lib/python3.11/dist-packages (2 Requirement already satisfied: numpy>=1.7 in /usr/local/lib/python3.11/dist-packages Requirement already satisfied: pandas>=0.17 in /usr/local/lib/python3.11/dist-package Requirement already satisfied: requests>=2.21.0 in /usr/local/lib/python3.11/dist-pac Requirement already satisfied: tqdm>=4.31.1 in /usr/local/lib/python3.11/dist-package Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.11/di Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-package Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dist-packa Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/ Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-package Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-p Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-p Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packages (f Requirement already satisfied: scikit-learn in /usr/local/lib/python3.11/dist-package Requirement already satisfied: numpy>=1.19.5 in /usr/local/lib/python3.11/dist-packag Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.11/dist-package Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.11/dist-packag Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.11/dist Requirement already satisfied: tensorflow in /usr/local/lib/python3.11/dist-packages Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.11/dist-packa Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.11/dist-pa Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.11/dist Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in /usr/local/lib/ Requirement already satisfied: google-pasta>=0.1.1 in /usr/local/lib/python3.11/dist- Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.11/dist-pac Requirement already satisfied: opt-einsum>=2.3.2 in /usr/local/lib/python3.11/dist-pa Requirement already satisfied: packaging in /usr/local/lib/python3.11/dist-packages (Requirement already satisfied: protobuf!=4.21.0,!=4.21.1,!=4.21.2,!=4.21.3,!=4.21.4,! Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.11/dist- Requirement already satisfied: setuptools in /usr/local/lib/python3.11/dist-packages Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.11/dist-packages Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.11/dist-pac Requirement already satisfied: typing-extensions>=3.6.6 in /usr/local/lib/python3.11/ Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.11/dist-packag Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.11/dist- Requirement already satisfied: tensorboard<2.19,>=2.18 in /usr/local/lib/python3.11/d Requirement already satisfied: keras>=3.5.0 in /usr/local/lib/python3.11/dist-package Requirement already satisfied: numpy<2.1.0,>=1.26.0 in /usr/local/lib/python3.11/dist Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.11/dist-package

Requirement already satisfied: ml-dtypes<0.5.0,>=0.4.0 in /usr/local/lib/python3.11/d Requirement already satisfied: tensorflow-io-gcs-filesystem>=0.23.1 in /usr/local/lib Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.11/dist-p Requirement already satisfied: rich in /usr/local/lib/python3.11/dist-packages (from Requirement already satisfied: namex in /usr/local/lib/python3.11/dist-packages (from Requirement already satisfied: optree in /usr/local/lib/python3.11/dist-packages (fro Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/ Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-package Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-p Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-p Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.11/dist-pack Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/li Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.11/dist-pack Requirement already satisfied: MarkupSafe>=2.1.1 in /usr/local/lib/python3.11/dist-pa Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.11/dis Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.11/d Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.11/dist-packages

* ## Mount path

```
from google.colab import drive
# Mount your Google Drive
drive.mount('/content/drive', force_remount=True)
```

Mounted at /content/drive

```
import os
# Check your Google Drive contents
drive_path = '/content/drive/My Drive/'  #driver route
# List contents of the drive root
os.listdir(drive_path)
```

['2)Bioinformatics_work', 'my_projects',
'1)Molecular_collection',
'phd_preparation',
'Bulk cell transcriptomic data analysis.gdoc',
'nr14_old_drive',
'COLLECTIONS',
'Colab Notebooks',
'PYTHON',
'Status report.gslides', 'Science
project.gslides', 'Untitled
presentation.gslides', 'REJWANA',
'CV']

Load count data

```
import gzip

geo_count_path = '/content/drive/My
Drive/REJWANA/2_my_work/8_AI_ML_Deep/GSE5281_multi_omics_deep_learning/data/GSE5281_series_matri
x.txt.gz'
```

```
with gzip.open(geo_count_path, 'rt') as f:
    geo_text = f.read()
```

```
print(geo_text[:900])  # print the first 900 characters
```

```
!Series_title	"Alzheimer's disease and the normal aged brain (steph-affy-human-4337
!Series_geo_accession	"GSE5281"
!Series_status	"Public on Jul 10 2006"
!Series_submission_date	"Jul 10 2006"
!Series_last_update_date	"Nov 08 2019"
!Series_pubmed_id	"17077275"
!Series_pubmed_id	"18332434"
!Series_pubmed_id	"29937276"
!Series_pubmed_id	"18270320"
!Series_summary	"Information about the genes that are preferentially expressed during
```

We need to manually pre-process the count data by removing unnecessary rows.:

---

* ## Load clean count data

```
clean_count_data = open('/content/drive/My
Drive/REJWANA/2_my_work/8_AI_ML_Deep/GSE5281_multi_omics_deep_learning/data/clean_GSE5281_series
_matrix.txt').read()
```

```
print(clean_count_data[:9000])  # print the first 9000 characters
```

```
"ID_REF"	"GSM119615"	"GSM119616"	"GSM119617"	"GSM119618"	"GSM1
"1007_s_at"	693.9733	807.14514	1722.625	1668.0575	2231.
"1053_at"	9.755318	19.05227	13.060389	14.6522455	138.6
"117_at"	13.346846	18.217276	97.42547	98.197945	106.4
"121_at"	173.09245	289.3154	537.1828	496.34656	477.8
"1255_g_at"	52.61695	110.375984	99.021736	17.306772	628.8
"1294_at"	67.753624	96.7
```

```
import pandas as pd
```

N Rejwan

```
raw_metadata = pd.read_excel('/content/drive/My Drive/REJWANA/2_my_work/8_AI_ML_Deep/GSE5281_multi_omics_deep_learning/data/raw_metadata_GSE5281_sample_characteristics.xls')
```

```
raw_metadata.head(10)   # Shows first 10 rows
```

	GEO Accession:	Sample Name:	MAGE Identifier:	Sample Description:	Sample Amount:	Bio-Source Name:	Organism:	Organ/Tissue Type:	Organ Region:	Cell Type:	Ethnicity:	Developmental Stage:	Disease State:	Sex:	Genetic Variation:	Age:
0	GSM119615	EC control 1	arrayconsortium.tgen.org:biosample.433887	AD control	10 ug	EC control 1	Human	brain	Entorhinal Cortex	layer III neurons	Caucasian	Adult	normal	male	None	63 years
1	GSM119616	EC control 2	arrayconsortium.tgen.org:biosample.433901	AD control	10 ug	EC control 2	Human	brain	Entorhinal Cortex	layer III neurons	Caucasian	Adult	normal	male	None	85 years
2	GSM119617	EC control 3	arrayconsortium.tgen.org:biosample.433915	AD control	10 ug	EC control 3	Human	brain	Entorhinal Cortex	layer III neurons	Caucasian	Adult	normal	male	None	80 years
3	GSM119618	EC control 4	arrayconsortium.tgen.org:biosample.433929	AD control	10 ug	EC control 4	Human	brain	Entorhinal Cortex	layer III neurons	Caucasian	Adult	normal	male	None	80 years
4	GSM119619	EC control 5	arrayconsortium.tgen.org:biosample.433943	AD control	10 ug	EC control 5	Human	brain	Entorhinal Cortex	layer III neurons	Caucasian	Adult	normal	female	None	102 years
5	GSM119620	EC control 6	arrayconsortium.tgen.org:biosample.433957	AD control	10 ug	EC control 6	Human	brain	Entorhinal Cortex	layer III neurons	Caucasian	Adult	normal	male	None	79 years
6	GSM119621	EC control 7	arrayconsortium.tgen.org:biosample.433971	AD control	10 ug	EC control 7	Human	brain	Entorhinal Cortex	layer III neurons	Caucasian	Adult	normal	male	None	76 years
7	GSM119622	EC control 8	arrayconsortium.tgen.org:biosample.433985	AD control	10 ug	EC control 8	Human	brain	Entorhinal Cortex	layer III neurons	Caucasian	Adult	normal	male	None	83 years
8	GSM119623	EC control 9	arrayconsortium.tgen.org:biosample.433999	AD control	10 ug	EC control 9	Human	brain	Entorhinal Cortex	layer III neurons	Caucasian	Adult	normal	male	None	79 years

N. Rejwan

9	GSM119624	EC control 10	arrayconsortium.tgen.org::biosample.434013	AD control	10 ug	EC control 10	Human	brain	Entorhinal Cortex	layer III neurons	Caucasian	Adult	normal	female	None	88 years

```
clean_metadata = pd.read_excel('/content/drive/My Drive/REJWANA/2_my_work/8_AI_ML_Deep/GSE5281_multi_omics_deep_learning/data/processed_metadata_GSE5281.xls')
```

```
clean_metadata.head(10)   # Shows first 10 rows
```

	GEO Accession:	Sample Name:	Sample Description:	Organ Region:	Cell Type:	Disease State:	Sex:	Age:
0	GSM119615	EC control 1	AD control	Entorhinal Cortex	layer III neurons	normal	male	63 years
1	GSM119616	EC control 2	AD control	Entorhinal Cortex	layer III neurons	normal	male	85 years
2	GSM119617	EC control 3	AD control	Entorhinal Cortex	layer III neurons	normal	male	80 years
3	GSM119618	EC control 4	AD control	Entorhinal Cortex	layer III neurons	normal	male	80 years
4	GSM119619	EC control 5	AD control	Entorhinal Cortex	layer III neurons	normal	female	102 years
5	GSM119620	EC control 6	AD control	Entorhinal Cortex	layer III neurons	normal	male	79 years
6	GSM119621	EC control 7	AD control	Entorhinal Cortex	layer III neurons	normal	male	76 years
7	GSM119622	EC control 8	AD control	Entorhinal Cortex	layer III neurons	normal	male	83 years
8	GSM119623	EC control 9	AD control	Entorhinal Cortex	layer III neurons	normal	male	79 years
9	GSM119624	EC control 10	AD control	Entorhinal Cortex	layer III neurons	normal	female	88 years

Here, clean_count_data = gene expression data as a .txt ﬁle (read into a variable) clean_metadata

= processed metadata loaded as a DataFrame from Excel

```
txt_path = '/content/drive/My
Drive/REJWANA/2_my_work/8_AI_ML_Deep/GSE5281_multi_omics_deep_learning/data/clean_GSE5281_series
_matrix.txt'

# Skip metadata lines (usually starting with '!')
expr_df = pd.read_csv(txt_path, sep='\t', comment='!', index_col=0)
```

```
# Transpose: samples = rows, genes = columns
X_df = expr_df.T
print("Expression shape (samples x genes):", X_df.shape)
X_df.head()
```

Expression shape (samples x genes): (161, 54675)

ID_REF	1007_s_at	1053_at	117_at	121_at	1255_g_at	1294_at	13
GSM119615	693.97330	9.755318	13.346846	173.09245	52.616950	67.753624	209
GSM119616	807.14514	19.052270	18.217276	289.31540	110.375984	96.780740	470.
GSM119617	1722.62500	13.060389	97.425470	537.18280	99.021736	76.282050	908
GSM119618	1668.05750	14.652245	98.197945	496.34656	17.306772	189.819600	1029.
GSM119619	2231.29690	138.676830	106.468440	477.86517	628.816160	276.476560	1366

5 rows × 54675 column

N. Rejwan

* loading and processing the gene expression count data

```
import pandas as pd

count_data_txt = '/content/drive/My
Drive/REJWANA/2_my_work/8_AI_ML_Deep/GSE5281_multi_omics_deep_learning/data/clean_GSE5281_series
_matrix.txt'
```

```
count_data_csv = pd.read_csv(count_data_txt, sep='\t', comment='!', index_col=0)
```

```
print("Expression shape (samples x genes):", count_data_csv.shape)
```

```
# Transpose the data (samples as rows, genes as columns)
X_df = count_data_csv.T
print("Transposed shape (samples x genes):", X_df.shape)
```

```
X_df.head(10)
```

Expression shape (samples x genes): (54675, 161)
Transposed shape (samples x genes): (161, 54675)

ID_REF	1007_s_at	1053_at	117_at	121_at	1255_g_at	1294_at	13
GSM119615	693.97330	9.755318	13.346846	173.09245	52.616950	67.753624	209.
GSM119616	807.14514	19.052270	18.217276	289.31540	110.375984	96.780740	470
GSM119617	1722.62500	13.060389	97.425470	537.18280	99.021736	76.282050	908.
GSM119618	1668.05750	14.652245	98.197945	496.34656	17.306772	189.819600	1029
GSM119619	2231.29690	138.676830	106.468440	477.86517	628.816160	276.476560	1366.
GSM119620	990.54100	7.186742	98.710520	355.09200	105.254600	130.088290	624.
GSM119621	1396.82420	10.170890	164.326660	467.56790	142.269180	131.513340	1589.
GSM119622	1026.05650	34.056934	46.306164	294.58328	123.016136	109.670810	858.
GSM119623	736.77030	7.829473	35.067910	224.52406	183.396910	63.197280	643.

N. Rejwana

GSM119624	938.81710	40.235332	72.864900	298.28473	180.814250	78.263855	412.

10 rows × 54675 columns

* ## Merge with Metadata (to get labels)

```
print(clean_metadata.columns)
```

```
from sklearn.preprocessing import StandardScaler

merged_df = X_df.merge(clean_metadata, left_index=True, right_on='GEO Accession:')
```

```
metadata_cols = ['GEO Accession:', 'Disease State:']
gene_columns = [col for col in merged_df.columns if col not in metadata_cols]
```

```
X = merged_df[gene_columns].copy()
X = X.apply(pd.to_numeric, errors='coerce').fillna(0)
```

```
# Extract labels (Alzheimer’s = 1, Normal = 0)
y = merged_df['Disease State:'].apply(lambda x: 1 if 'alzheimer' in str(x).lower() else
0).values
```

```
# Normalize the data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
print(" X_scaled shape:", X_scaled.shape)
print("y shape:", y.shape)
```

X_scaled shape: (161, 54681)
y shape: (161,)

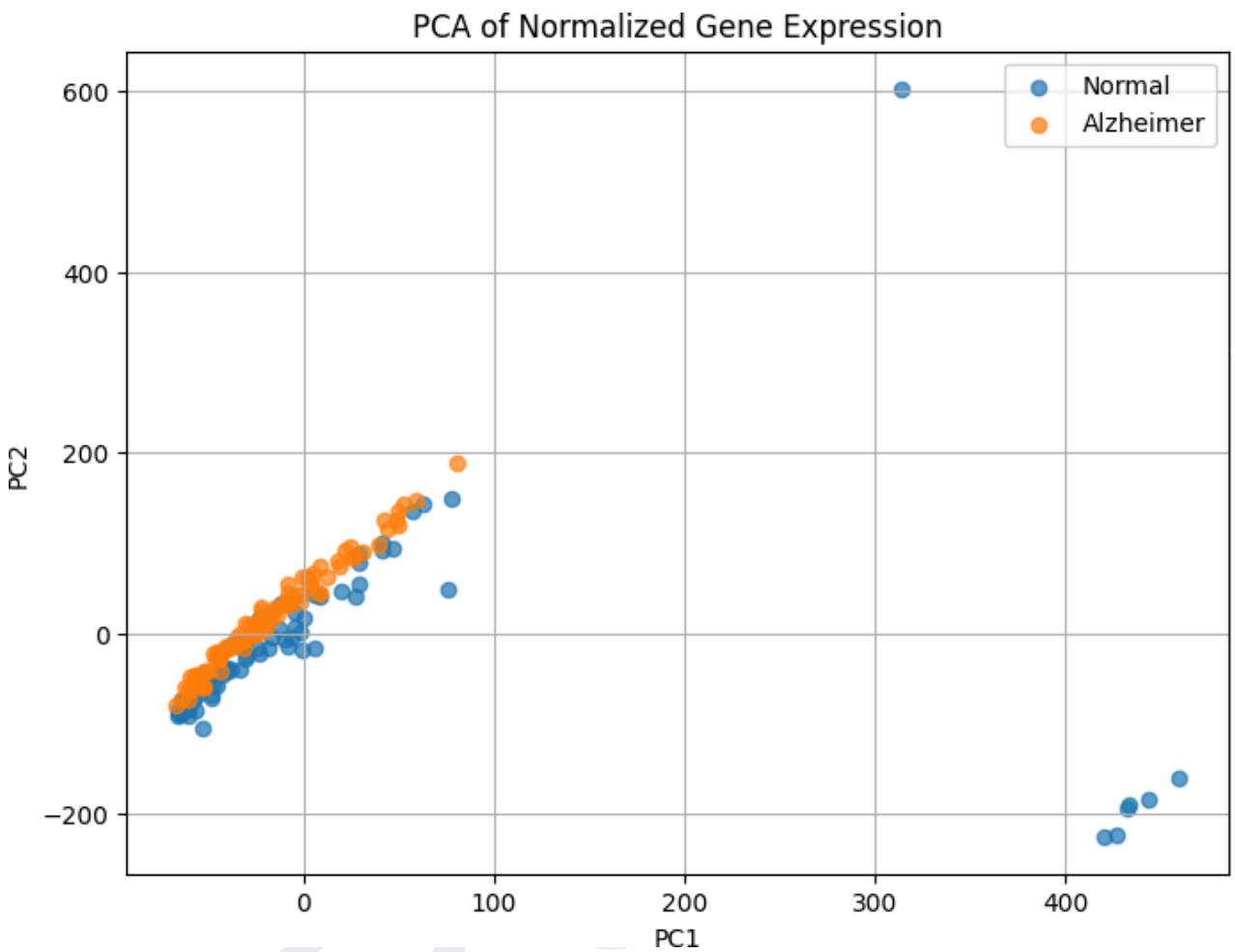
Plot PCA

```
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

# Perform PCA to reduce to 2 components
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
```

```
# Plot the PCA result
plt.figure(figsize=(4, 3))
plt.scatter(X_pca[y == 0, 0], X_pca[y == 0, 1], label='Normal', alpha=0.7)
plt.scatter(X_pca[y == 1, 0], X_pca[y == 1, 1], label='Alzheimer', alpha=0.7)
plt.title('PCA of Normalized Gene Expression')
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.legend()
```

```
plt.grid(True)
plt.show()
```

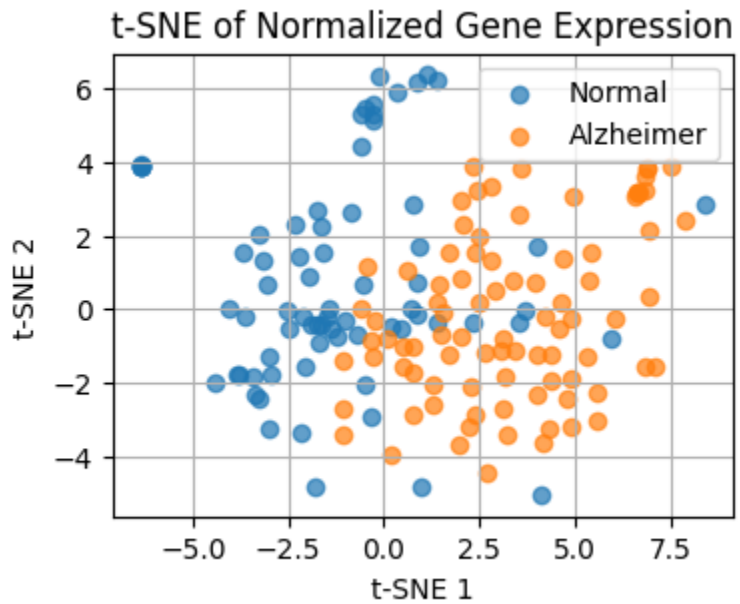


t-SNE

```
from sklearn.manifold import TSNE

# Run t-SNE on a sample of the data
tsne = TSNE(n_components=2, perplexity=30, random_state=42)
X_tsne = tsne.fit_transform(X_scaled)
```

```
# Plot
plt.figure(figsize=(4, 3))
plt.scatter(X_tsne[y == 0, 0], X_tsne[y == 0, 1], label='Normal', alpha=0.7)
plt.scatter(X_tsne[y == 1, 0], X_tsne[y == 1, 1], label='Alzheimer', alpha=0.7)
plt.title('t-SNE of Normalized Gene Expression')
plt.xlabel('t-SNE 1')
plt.ylabel('t-SNE 2')
plt.legend()
plt.grid(True)
plt.show()
```

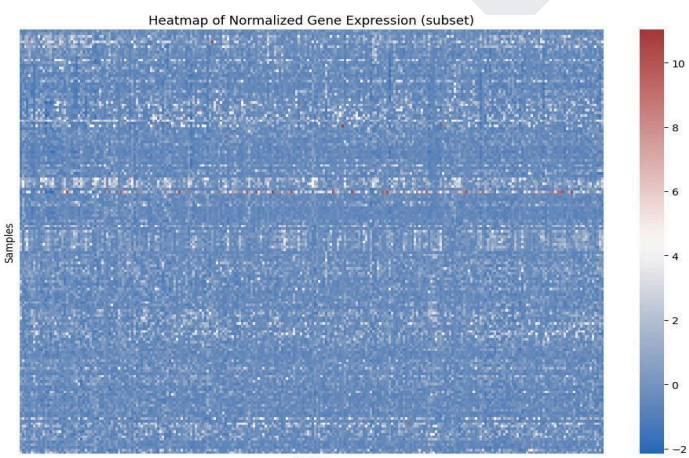


Heatmap

```
import seaborn as sns


subset = X_scaled[:250, :250]  # First 250 samples & 250 genes
```

```
plt.figure(figsize=(4, 3))
sns.heatmap(subset, cmap='vlag', xticklabels=False, yticklabels=False)
plt.title('Heatmap of Normalized Gene Expression (subset)')
plt.xlabel('Genes')
plt.ylabel('Samples')
plt.show()
```



Split data after normalization

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)
```

```
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA


pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
```

```
X_train_pca, X_test_pca = X_pca[:len(X_train)], X_pca[len(X_train):]
```

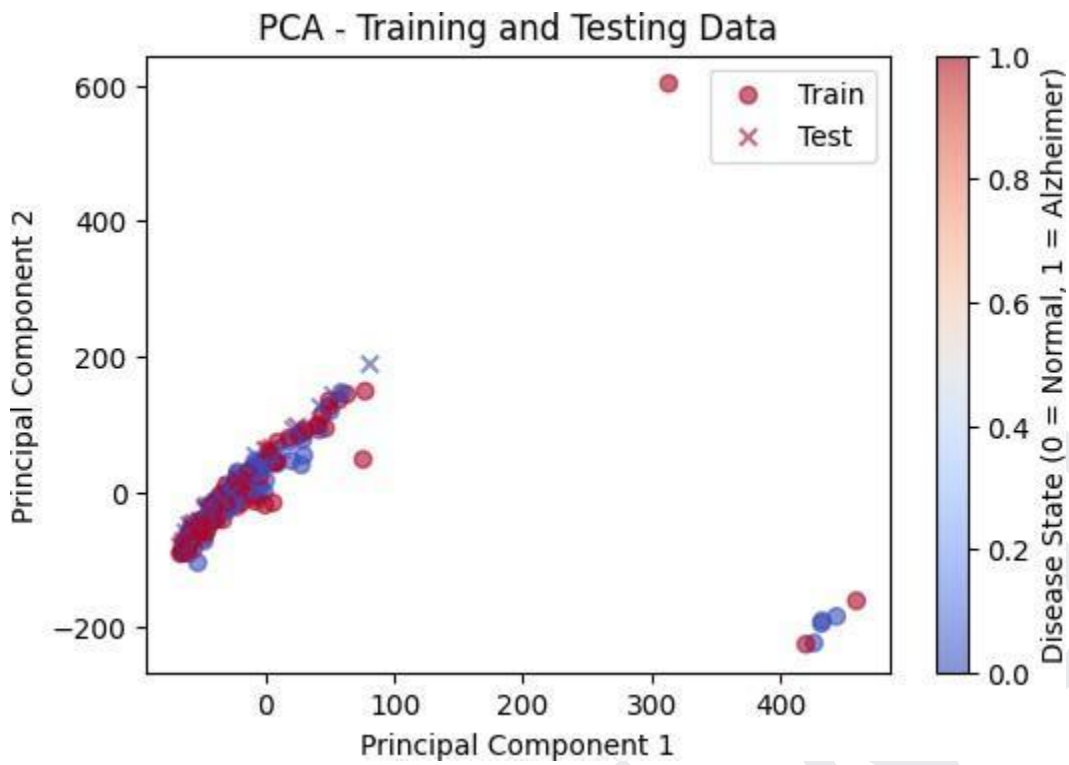
```
plt.figure(figsize=(4, 3))
```

```
plt.scatter(X_train_pca[:, 0], X_train_pca[:, 1], c=y_train, cmap='coolwarm', label='Train',
alpha=0.6)
```

```
plt.scatter(X_test_pca[:, 0], X_test_pca[:, 1], c=y_test, cmap='coolwarm', marker='x',
label='Test', alpha=0.6)
```

```
plt.title('PCA - Training and Testing Data')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend()
plt.colorbar(label='Disease State (0 = Normal, 1 = Alzheimer)')
```

```
# Show the plot
plt.show()
```



Feedforward Neural Network (FNN)

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.optimizers import Adam

# 1. Define the Feedforward Neural Network
model_fnn = Sequential([
    Dense(128, activation='relu', input_dim=X_train.shape[1]),  # Input layer
    Dropout(0.5),
    Dense(64, activation='relu'),                                # Hidden layer
    Dropout(0.5),
    Dense(1, activation='sigmoid')                               # Output layer for binary
classification
])
```

```
# 2. Compile the model
model_fnn.compile(optimizer=Adam(learning_rate=0.001),
                  loss='binary_crossentropy',
                  metrics=['accuracy'])
```

```
# 3. View model architecture
model_fnn.summary()
```

```
# 4. Train the model
history_fnn = model_fnn.fit(
    X_train, y_train,
    epochs=20,
    batch_size=32,
    validation_split=0.2,
    verbose=1
)
```

Layer (type)	Output Shape	Param #
dense_3 (Dense)	(None, 128)	6,999,296
dropout_2 (Dropout)	(None, 128)	0
dense_4 (Dense)	(None, 64)	8,256
dropout_3 (Dropout)	(None, 64)	0
dense_5 (Dense)	(None, 1)	65

Total params: 7,007,617 (26.73 MB)
Trainable params: 7,007,617 (26.73 MB)
Non-trainable params: 0 (0.00 B)

```
Epoch 1/204/4 ━━━━━━━━━━━━━━━━━━━━ 3s 175ms/step - accuracy: 0.6641 -
loss: 1.5237 - val_accuracy: 0.9615 - val_loss: 0.3151
Epoch 2/204/4 ━━━━━━━━━━━━━━━━━━━━ 0s 106ms/step - accuracy: 0.6683 -
loss: 3.6900 - val_accuracy: 0.8846 - val_loss: 1.5593
Epoch 3/204/4 ━━━━━━━━━━━━━━━━━━━━ 1s 106ms/step - accuracy: 0.7814 -
loss: 3.7669 - val_accuracy: 0.8846 - val_loss: 1.1554
Epoch 4/204/4 ━━━━━━━━━━━━━━━━━━━━ 0s 114ms/step - accuracy: 0.9241 -
loss: 0.9723 - val_accuracy: 0.8846 - val_loss: 0.4878
Epoch 5/204/4 ━━━━━━━━━━━━━━━━━━━━ 1s 99ms/step - accuracy: 0.8779 -
loss: 3.9484 - val_accuracy: 0.9615 - val_loss: 0.0397
Epoch 6/204/4 ━━━━━━━━━━━━━━━━━━━━ 1s 108ms/step - accuracy: 0.8902 -
loss: 1.7506 - val_accuracy: 0.9615 - val_loss: 0.1419
Epoch 7/204/4 ━━━━━━━━━━━━━━━━━━━━ 1s 110ms/step - accuracy: 0.9019 -
loss: 1.5040 - val_accuracy: 0.8846 - val_loss: 2.5394
Epoch 8/204/4 ━━━━━━━━━━━━━━━━━━━━ 1s 169ms/step - accuracy: 0.9301 -
loss: 0.7997 - val_accuracy: 0.8846 - val_loss: 3.5826
Epoch 9/204/4 ━━━━━━━━━━━━━━━━━━━━ 1s 154ms/step - accuracy: 0.8902 -
loss: 3.0192 - val_accuracy: 0.8846 - val_loss: 2.5461
Epoch 10/204/4 ━━━━━━━━━━━━━━━━━━━━ 1s 142ms/step - accuracy: 0.9207 -
loss: 2.1202 - val_accuracy: 0.9615 - val_loss: 0.3449
Epoch 11/204/4 ━━━━━━━━━━━━━━━━━━━━ 1s 146ms/step - accuracy: 0.8996 -
loss: 4.3672 - val_accuracy: 0.9615 - val_loss: 0.1282
Epoch 12/204/4 ━━━━━━━━━━━━━━━━━━━━ 1s 154ms/step - accuracy: 0.9510 -
loss: 2.7508 - val_accuracy: 0.9615 - val_loss: 0.2762
Epoch 13/204/4 ━━━━━━━━━━━━━━━━━━━━ 1s 152ms/step - accuracy: 0.9450 -
loss: 1.6111 - val_accuracy: 0.9615 - val_loss: 0.6793
Epoch 14/204/4 ━━━━━━━━━━━━━━━━━━━━ 0s 117ms/step - accuracy: 0.9045 -
loss: 1.1012 - val_accuracy: 0.8846 - val_loss: 1.6726
Epoch 15/204/4 ━━━━━━━━━━━━━━━━━━━━ 1s 105ms/step - accuracy: 0.9390 -
loss: 2.8538 - val_accuracy: 0.8846 - val_loss: 1.4096
Epoch 16/204/4 ━━━━━━━━━━━━━━━━━━━━ 0s 108ms/step - accuracy: 0.9909 -
loss: 0.1003 - val_accuracy: 0.8846 - val_loss: 1.3204
Epoch 17/204/4 ━━━━━━━━━━━━━━━━━━━━ 0s 107ms/step - accuracy: 0.9439 -
loss: 0.5428 - val_accuracy: 0.8846 - val_loss: 1.3982
```

N. Rejwan

```
Epoch 18/204/4 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1s 104ms/step - accuracy: 0.9718 -
loss: 2.6047 - val_accuracy: 0.9231 - val_loss: 1.5857
Epoch 19/204/4 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1s 102ms/step - accuracy: 0.9447 -
loss: 0.6609 - val_accuracy: 0.8846 - val_loss: 1.9323
Epoch 20/204/4 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1s 101ms/step - accuracy: 0.9356 -
loss: 1.2823 - val_accuracy: 0.8846 - val_loss: 2.0200
```

Evaluate the model

```
test_loss, test_acc = model_fnn.evaluate(X_test, y_test)
print(f"Test accuracy: {test_acc:.4f}")
print(f"Test loss: {test_loss:.4f}")
```

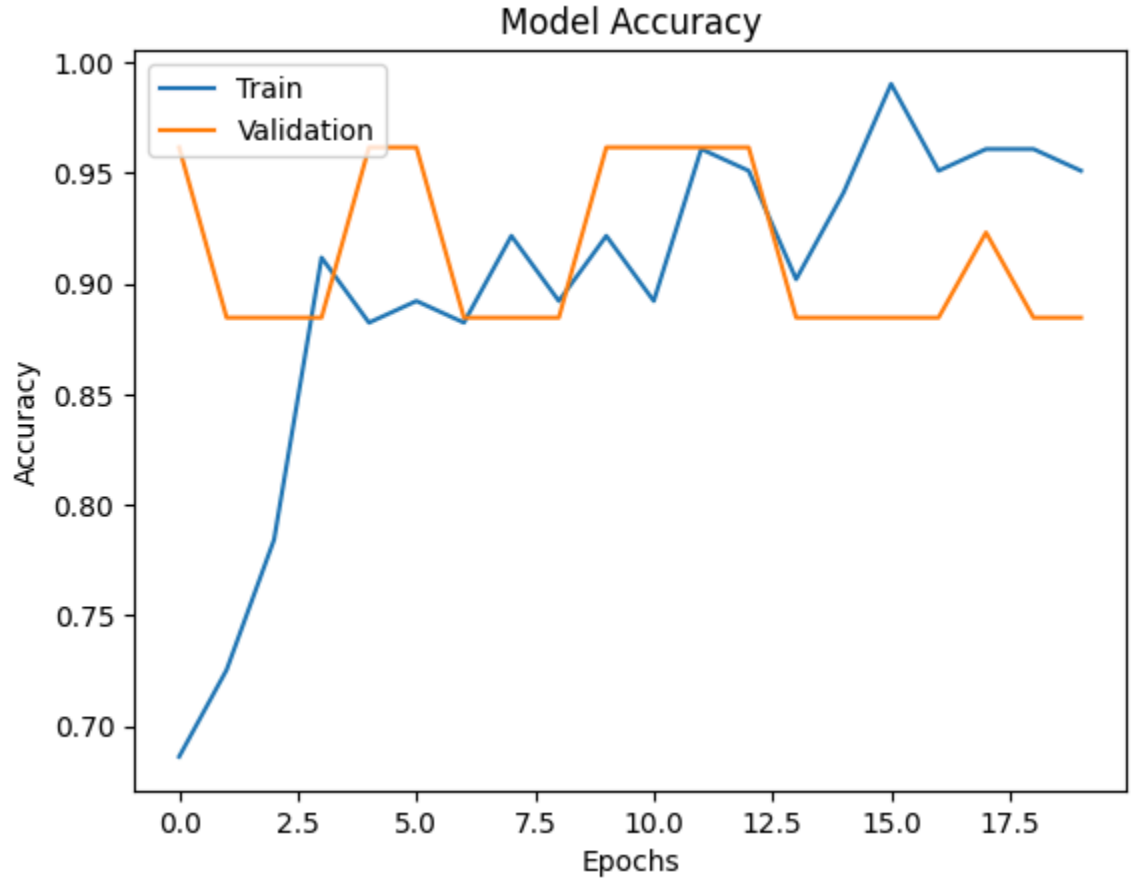
```
2/2 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - accuracy: 0.9081 - loss: 2.8950
Test accuracy: 0.9091
Test loss: 2.8651
```

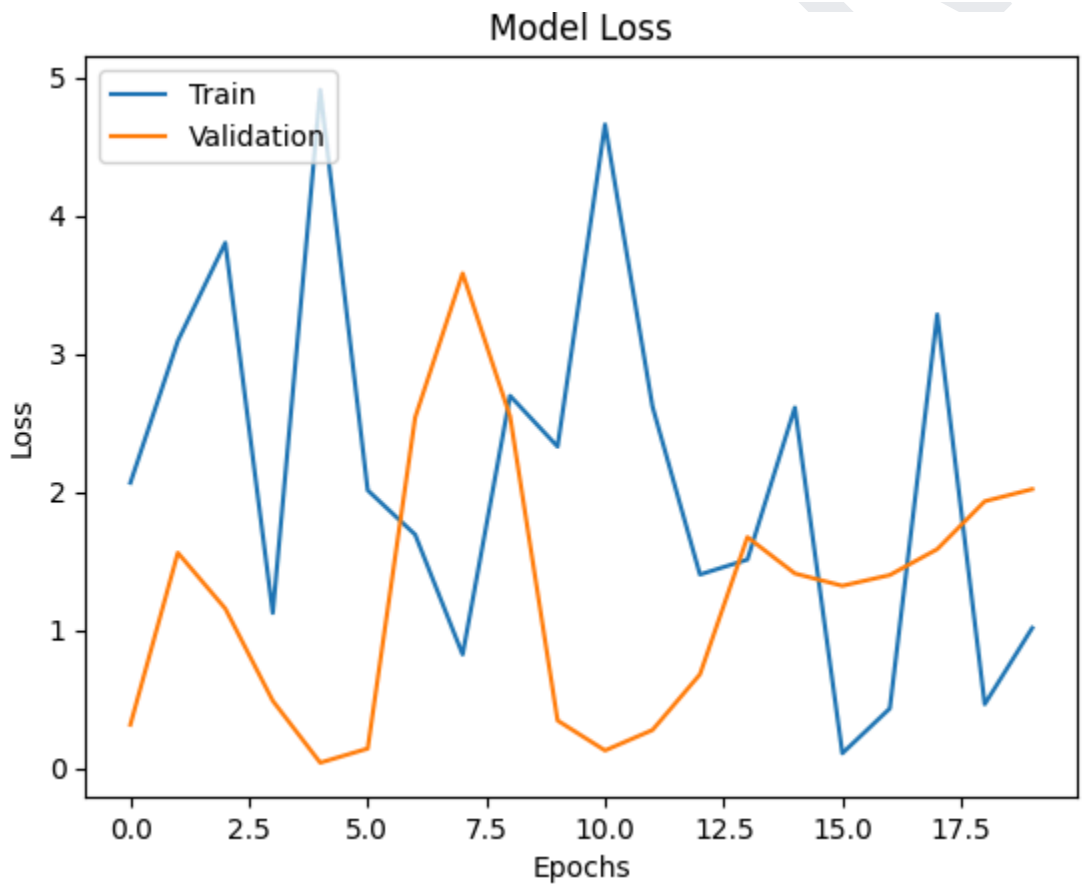
Plot the training and validation loss/accuracy

```
import matplotlib.pyplot as plt

# Plot training & validation accuracy values
plt.plot(history_fnn.history['accuracy'])
plt.plot(history_fnn.history['val_accuracy'])
plt.title('Model Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend(['Train', 'Validation'], loc='upper left')
plt.show()
```

```
# Plot training & validation loss values
plt.plot(history_fnn.history['loss'])
plt.plot(history_fnn.history['val_loss'])
plt.title('Model Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend(['Train', 'Validation'], loc='upper left')
plt.show()
```


Model Accuracy
Train
Validation
Accuracy
Epochs
1.00
0.95
0.90
0.85
0.80
0.75
0.70
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5


Model Loss
Train
Validation
Loss
Epochs
5
4
3
2
1
0
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5

* Make predictions

```
# predictions on the test data
y_pred = model_fnn.predict(X_test)

y_pred_binary = (y_pred > 0.5).astype(int)  # Convert the predictions to binary (0 or 1) based
on a threshold (0.5)
```

```
from sklearn.metrics import confusion_matrix, classification_report  # Compare the predicted
values with actual labels
```

```
conf_matrix = confusion_matrix(y_test, y_pred_binary)   # Confusion Matrix
print(f"Confusion Matrix:\n{conf_matrix}")
```

```
class_report = classification_report(y_test, y_pred_binary)  # Classification Report
print(f"Classification Report:\n{class_report}")
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 75ms/step
Confusion Matrix:
[[14  1]
 [ 2 16]]
Classification Report:
              precision    recall  f1-score   support

           0       0.88      0.93      0.90        15
           1       0.94      0.89      0.91        18

    accuracy                           0.91        33
   macro avg       0.91      0.91      0.91        33
weighted avg       0.91      0.91      0.91        33
```

N. Rejwan

* ROC Curve

```
import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve, auc


y_pred_prob = model_fnn.predict(X_test)  # Get predicted probabilities (not binary predictions)
```

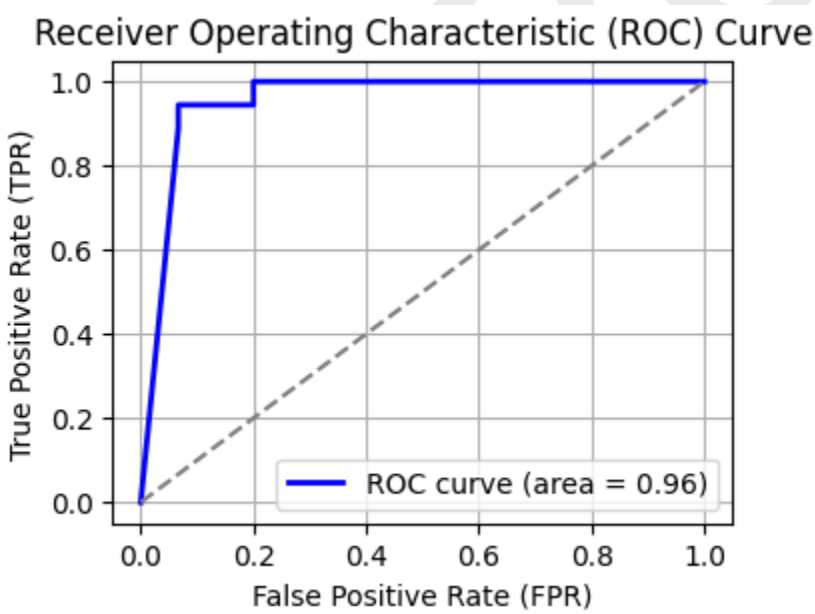
```
fpr, tpr, thresholds = roc_curve(y_test, y_pred_prob) #  Calculate the ROC curve
```

```
roc_auc = auc(fpr, tpr) # Calculate the Area Under the Curve (AUC)
```

```
# Plot the ROC curve
plt.figure(figsize=(4, 3))
plt.plot(fpr, tpr, color='b', lw=2, label=f'ROC curve (area = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')  # Diagonal line (no discrimination)
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.xlabel('False Positive Rate (FPR)')
plt.ylabel('True Positive Rate (TPR)')
plt.legend(loc='lower right')
plt.grid(True)
plt.show()
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step
```

Receiver Operating Characteristic (ROC) Curve



```
# Save the model
model_fnn.save('/content/drive/My
Drive/REJWANA/2_my_work/8_AI_ML_Deep/GSE5281_multi_omics_deep_learning/saved_model_fnn.h5')
```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`.

```
##Loaded model later
from tensorflow.keras.models import load_model

# Load the saved model
loaded_model = load_model('/content/drive/My
Drive/REJWANA/2_my_work/8_AI_ML_Deep/GSE5281_multi_omics_deep_learning/saved_model_fnn.h5')
```

```
# Check the model summary
loaded_model.summary()
```

WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile_metrics` will be empty until you train or evaluate the model.

Layer (type)	Output Shape	Param #
dense_3 (Dense)	(None, 128)	6,999,296
dropout_2 (Dropout)	(None, 128)	0
dense_4 (Dense)	(None, 64)	8,256
dropout_3 (Dropout)	(None, 64)	0
dense_5 (Dense)	(None, 1)	65

Total params: 7,007,619 (26.73 MB)
Trainable params: 7,007,617 (26.73 MB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 2 (12.00 B)